
AI-Enhanced F1 Simulator Individual Report

Author:

JUNJIE

PENG

Supervisor:

RUZANNA CHITCHYAN



Department of Computer Science

UNIVERSITY OF BRISTOL

A thesis submitted to the University of Bristol in accordance
with the requirements of the degree of MASTER OF SCIENCE
in xxx (major)

SEPTEMBER 2026

Word count: 4957

Abstract

This project asked whether IBM Granite could advise an autonomous TORCS driver without being placed inside its real-time control loop. I took responsibility for the bot's technical foundation: the SCR protocol layer, the initial low-level controller and safety layer, the original track model, Granite integration, replay and latency-evaluation tools, HPC deployment, and the strategy information shown on the dashboard. My teammate later developed the driving-behaviour tuning, opponent avoidance, crash recovery, pit handling, and most of the final test suite; I refer to those parts only as context for the integrated system.

The defining constraint was timing. TORCS expects a command approximately every 20 milliseconds, while Granite took 6.7 to 7.6 seconds to respond in the final live tests. Rather than asking the model to drive the car directly, I separated the fast controller from the slower advisor. The controller never waits for Granite, and model advice must pass through a local safety layer before it can affect the car.

My evaluation process changed as the project developed. I began by comparing complete races, but found that this was slow and gave each prompt different situations. I therefore built an offline replay tool that presents prompts with the same recorded vehicle states. This report concentrates on the decisions behind that architecture and evaluation method, including two measurements I later rejected and an early interpretation of strategy diversity that I withdrew. The most important experiment still missing is a controlled comparison of the same controller with and without Granite advice.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Junjie Peng

August 2026

Contents

Abstract	1
Declaration	2
1 Introduction and Context	4
2 Example of Innovation and Creativity	5
3 Technical Development	5
3.1 Building the SCR and Control Foundation	5
3.2 Keeping a Seven-Second Model out of a 20 ms Loop	6
3.3 Turning prose into a control decision	7
3.4 Giving the controller a view beyond its sensors	7
3.5 Moving Inference to BluePebble	8
4 Evaluation Skills	8
4.1 Questions Behind the Measurements	8
4.2 What the measurements do—and do not—show	9
4.3 Checking the instrument, not only the result	9
4.4 Discarding measurements that did not answer the question	10
4.5 Revising my own conclusion	10
5 Project Management Skills	10
5.1 Building the feature from the bottom up	10
5.2 Using Git as an engineering record	11
5.3 Explaining decisions to the team and client	11
5.4 Managing risk without stopping development	11
6 Critical Self-Evaluation and Future Work	11
6.1 What I would do differently	11
6.2 Future work	12
6.3 Closing reflection	12

1 Introduction and Context

Our group project connects TORCS 1.3.7, an open-source racing simulator, to IBM Granite through an OpenAI-compatible API [4]. The complete system combines a conversational racing engineer, live telemetry, event-driven commentary, and an autonomous bot that can receive strategic advice from Granite.

Within this system, I built the main technical path used by the driving bot. I began with the SCR state parser and control serializer, then extended the reference `snakeoil` client into the initial steering and speed controller. I later migrated these components into the main repository together with the first safety layer. On top of this foundation, I developed the original track model and lookahead structure, implemented the Granite strategy module, deployed the model service on BluePebble, and built the replay and trace-analysis tools used in my experiments. These components formed the path from simulator telemetry to local control and, separately, from recorded race states to Granite advice.

My teammate later concentrated on driving-behaviour tuning, opponent avoidance, crash recovery, pit handling, and most of the final regression suite. I refer to those features only where they affected the integration of my work. This distinction is important because the completed bot contains work from both developers, while this report evaluates my own contribution.

The main technical difficulty was deciding where a slow and uncertain model could fit within a controller that was expected to operate in real time. TORCS requests steering, throttle, brake, and gear commands approximately every 20 milliseconds, whereas Granite took between 6.7 and 7.6 seconds to respond in the final live tests. A synchronous implementation would therefore cause the bot to miss hundreds of simulator updates while waiting for one strategy decision.

Previous work provided useful guidance on how these two timescales might be separated. SayCan places a language model above lower-level robot skills so that the model contributes high-level knowledge without directly executing every action [1]. Brooks similarly argues that lower-level behaviour should continue to operate when higher-level capabilities are added [5]. The Simplex architecture retains a dependable control path when a more complex component cannot be trusted [6]. These ideas supported my decision to use Granite as an adviser rather than as the vehicle controller.

The comparison has limits. My implementation is not a formal Simplex system, and the safety filter does not provide a mathematical guarantee for every possible vehicle state. Granite is also a general-purpose language model rather than a component designed for real-time racing. The practical objective was narrower: model latency, an invalid reply, or a network failure should not interrupt the existing local controller.

I addressed this by separating the system into two timing domains. The fast domain processes SCR telemetry and calculates the next control command. The slow domain periodically requests strategic advice from Granite and publishes only its latest valid decision. I then developed replay and latency tools so that the slow domain could be investigated without repeatedly running a complete race.

This architecture shows how Granite can be connected without blocking the controller, but it does not by itself show that Granite improves driving performance. Replay can compare prompt responses to the same recorded states, whereas a controlled Granite-enabled and Granite-disabled experiment is still needed to measure effects on lap time, consistency, damage, and off-track events. The following sections therefore separate the architecture I implemented from the conclusions that the available experiments can support.

2 Example of Innovation and Creativity

I began with what seemed the most convincing test: give each prompt a complete race and compare the result. It quickly became clear that this was also the least controlled test. A race took roughly 10 to 15 minutes, and a small early steering difference could change the traffic, collisions, and tyre conditions encountered later. After several hours, I could still be comparing prompts that had never faced the same situation.

That experience changed the way I thought about the simulator. For the strategy layer, TORCS was mainly a source of vehicle states. The strategy code could not tell whether a state came from a live race or a file. I therefore recorded the inputs passed to Granite and prepared the `race2`, `race4`, `race4_lowfuel`, and `race5` JSONL corpora, containing 2,870 states in total.

I then developed `bot_replay.py` to run the strategy layer against those records without starting TORCS. This turned a prompt comparison from two unrelated race reports into a paired experiment. The `-limit` option uses deterministic, evenly spaced sampling, so two variants given the same trace and limit receive exactly the same selected states. Granite itself is still variable at temperature 0.2, and stronger claims would require repeated runs or confidence intervals, but at least the simulator input no longer moves between comparisons. Replay made the model inputs easier to control, but the resulting traces still needed to be inspected after each run. I used the `race4` experiment as one of the main recorded cases and implemented `analyse_bot_trace.py` to process the bot traces after collection. Keeping trace collection and analysis separate meant that I could return to the same raw records when a metric or interpretation changed, rather than repeating the complete race or relying on what had been visible on the dashboard. A smaller version of the same reasoning arose when the dashboard appeared to be frozen on `ATTACK`. Making Granite choose more varied strategies would have improved the display by changing the behaviour being measured. I treated it as a presentation problem instead and implemented `display_label()`. The same strategy can appear as *Leading: holding pace*, *Chasing: full throttle*, or *Attacking*, depending on race context. Tests confirm that computing the label does not alter the value passed to the controller. This was a modest feature, but it protected a boundary I considered important: presentation should explain a decision, not quietly change it.

3 Technical Development

3.1 Building the SCR and Control Foundation

The first useful milestone was more basic than connecting Granite: the bot needed to receive a simulator state and return a valid control command on every cycle. I implemented the SCR state parser and control serializer as the two boundaries of this exchange. The parser converts the name-value groups received from TORCS into the internal vehicle state, while the serializer turns the controller output back into the format expected by the simulator. Keeping these operations outside the controller made it possible to change the driving logic without also changing the communication protocol.

I used the reference `snakeoil` client as the starting point for the initial low-level controller. I retained its basic connection mechanism, but implemented the steering and speed-control path needed by our bot. I first established a pure-pursuit baseline, then completed the remaining control steps so that parsed telemetry could produce steering, throttle, brake, and gear outputs through the serializer. At this stage, the purpose was not to optimise lap time. It was to produce a working local controller that could be tested before either the track model or Granite was introduced.

I initially developed these components in the separate `ai_bot` prototype. Once the complete path was working, I migrated the parser, serializer, controller, and first safety layer into the main repository. This gave the later track and strategy modules a common vehicle-state interface instead of allowing each feature to interpret SCR data independently. It also meant that a Granite failure could be handled above an already functioning local controller rather than inside the simulator protocol code.

This order became important later in the project. The track model could add lookahead without changing SCR communication, and the Granite layer could request a strategy without producing control commands directly. My teammate later tuned the driving behaviour and added opponent-, recovery-, and pit-related logic. Those additions extended the baseline controller but did not change the protocol and control foundation described here.

3.2 Keeping a Seven-Second Model out of a 20 ms Loop

The feature rests on two timing domains, shown in Figure 1. The fast domain receives an SCR packet, updates the track model, and computes steering, throttle, brake, and gear. It is local, deterministic, and independent of the network. The slow domain periodically describes the race to Granite and reduces its reply to one of five model-selectable strategies: `ATTACK`, `NORMAL`, `DEFEND`, `SAVE_FUEL`, or `PIT`.

I kept `BLOCK` out of Granite’s options. Blocking depends on the rapidly changing position of another car, so advice arriving several seconds later could be actively dangerous. This was an early example of a rule I used throughout the bot: allocate a decision according to how quickly it must be corrected, not according to how strategically important it sounds.

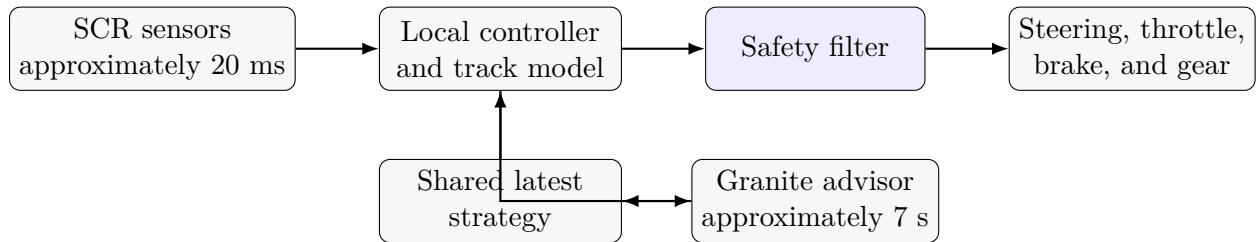


Figure 1: Separation of the fast controller and the slower Granite strategy service.

The essential rule is that the fast domain never waits for the slow one. Granite publishes its latest valid decision to a shared value; the controller reads that value when convenient. If the service is unreachable or its reply cannot be parsed, the car continues with the previous strategy. A stale strategy is usually manageable. A blocked control loop is not. This follows the broader separation between deliberative planning and reactive control discussed in embodied AI work [1], but the latency ratio here is unusually large.

Between the two domains, I implemented the initial `safety_filter`. Granite can request aggressive behaviour, but the local controller retains authority over the car’s physical limits. My teammate later added pit- and opponent-aware rules to the final version; those extensions are not presented here as my individual work.

I placed the map trust checks in one narrow component rather than scattering them through the controller. Table 1 records the five conditions. This made the safety claim easier to state and, more importantly, easier to test.

Table 1: Map trust conditions.

#	Condition	Reason
1	Odometer calibrated after a practice lap	Before calibration, accumulated map alignment error may affect lookahead.
2	Car is on the racing line	The map describes the intended line rather than every possible off-line path.
3	No opponent within 100 m ahead	The track map does not represent traffic.
4	Sensor range agrees with the map prediction	A disagreement may indicate an obstacle or traffic not represented in the map.
5	Local speed limit is at least 150 km/h	Map assistance has more benefit on faster sections and less safety margin near an apex.

3.3 Turning prose into a control decision

Connecting Granite involved more than sending an API request. The model returns an explanation in natural language, whereas the controller requires one value from a small set of permitted strategies. I implemented `middleware/bot_strategy.py` as the boundary between these two representations. The module constructs the request, submits it to the model service, interprets the reply, and makes only the latest valid strategy available to the rest of the bot. Keeping these responsibilities in one module prevented changes to the prompt or API from spreading into the low-level controller.

The prompt is assembled from the current speed, fuel level, damage, track position, race position, and gaps to nearby cars. I selected these inputs because they describe the longer-term race situation without asking Granite to reproduce the sensor-level calculations already performed by the local controller. The output is restricted to `ATTACK`, `NORMAL`, `DEFEND`, `SAVE_FUEL`, or `PIT`; fast behaviours whose correctness depends on the immediate position of another car remain local.

I made the parser tolerant about presentation but strict about output. It accepts several reasonable response formats, yet it publishes only a valid strategy value. If a reply remains ambiguous, the previous strategy is retained. Combined with the two-clock design, malformed prose therefore degrades to *no new advice* rather than a failed driving loop. The team suite covers prompt construction, parsing, safety integration, and failure handling; my own test work included updates to the Granite runtime and safety-integration tests.

3.4 Giving the controller a view beyond its sensors

The strategy layer was of little use if the local controller could not execute its advice safely. In early tests, the car carried too much speed into corners that were not yet clear in the short-range sensors. I developed the original `track_model.py` to represent the circuit as segments with curvature, length, and a derived speed limit. The controller could then brake for a corner before the sensors showed it clearly.

The initial speed calculation used a basic friction-circle approximation:

$$v = \sqrt{a_{\text{lat}} R}.$$

The first version used constant lateral acceleration. It kept the car safe, but made map-assisted driving unnecessarily slow in high-speed corners. I added a simplified downforce term, giving:

$$v^2 = \frac{a_{\text{lat}} R}{1 - k_{\text{ero}} a_{\text{lat}} R}.$$

A second error came from treating adjacent same-direction segments independently. The straightening correction then assigned 155 km/h to one 80 m-radius corner instead of 120 km/h. I fixed this by merging those segments before applying the correction. Braking points were propagated backwards over two laps, and I calibrated the odometer against measured lap distance to reduce accumulated map error. My teammate later tuned the final corner-speed parameters and added further chicane handling.

3.5 Moving Inference to BluePebble

Once the strategy layer was working, local inference became the slowest and least predictable part of my testing process. A response could take several seconds, and variation between runs made it difficult to tell whether a change came from the prompt, the simulator, or the local model service. I therefore moved Granite to Bristol’s BluePebble cluster. The purpose was not to place inference inside the real-time loop, but to give the slower strategy experiments a more consistent platform.

Running the model on BluePebble required more than submitting a single job. I wrote `serve_granite.sbatch` and `serve_granite_2080.sbatch` for the available deployment configurations, together with `preflight.sbatch` to check the environment before starting a longer service job. I also implemented `wait_ready.sh`, which waits for the model endpoint to become available, and `connect.sh`, which establishes the connection used by the bot. These scripts replaced a sequence of manual steps that had previously been easy to perform in the wrong order.

This work did not make Granite a real-time component. Responses on the cluster still took several seconds, so the two-clock architecture remained necessary. The practical improvement was experimental: the same service-startup process could be repeated for replay, prompt, and latency tests, and a failed readiness check could be separated from a slow model response. I later used this setup for the local-versus-HPC latency comparison reported in the evaluation section.

4 Evaluation Skills

4.1 Questions Behind the Measurements

My early evaluation collected outputs before clearly separating the questions they were intended to answer. This made some results look more informative than they were. Later, I reorganized the evidence around three questions: whether Granite could be isolated from the control loop, whether replay gave a fairer prompt comparison, and whether the selected strategies responded meaningfully to changes in race state.

Question	Evidence used	Main limitation
Can the controller continue without waiting for Granite?	Local control time compared with the 20 ms control-cycle target, together with measured Granite latency.	Timing isolation does not show that Granite improves driving performance.
Does replay give prompts comparable input states?	Deterministic sampling from the same recorded traces and a preliminary comparison of live and offline inference latency.	Replay is open-loop: a strategy cannot change the states encountered later.
Do prompt variants produce appropriate strategic changes?	Strategy outputs on shared recorded states, including the reasoning-prompt comparison.	The corpus mostly contains trouble-free racing, so ATTACK is often a reasonable response.

Table 2: Evaluation questions, evidence, and limits.

The following subsections examine these questions in detail and separate measurements produced by my work from results belonging to the integrated team system.

4.2 What the measurements do—and do not—show

The final group suite contains 17 test files covering SCR communication, control behaviour, recovery, track lookahead, traffic, latency, and Granite integration. On 18 August 2026, **pytest tests/bot** reported 247 passing tests and 16 additional parameterised subtests. Most of that suite was built by my teammate. My own additions were narrower: the dedicated 113-line test file for `display_label()` and updates to the Granite runtime and safety-integration tests.

For that reason, Table 3 includes only measurements directly connected to my work. It does not use the team’s recovery, pit, or fault-injection results as evidence of my individual contribution, and the figures should be read as measurements of the tested runs rather than general guarantees.

Table 3: Measurements related to my contribution.

Measure	Result
Control computation	Median 0.018 ms for the measured local computation, against a 20 ms control-cycle target
Granite latency	Local LM Studio median 9.325 s; HPC 6.9 to 7.2 s; live-race samples 6.7 to 7.6 s
Replay corpus	2,870 states across race2, race4, race4_lowfuel, and race5

Opponent avoidance, crash recovery, pit behaviour, and the majority of the final **pytest** suite were implemented and evaluated by my teammate. Their results are therefore not presented as evidence of my individual contribution.

4.3 Checking the instrument, not only the result

Because replay became the basis of my prompt comparison, I needed to test the instrument itself. I compared three live-race latencies—6.7, 7.2, and 7.6 seconds—with offline measurements of 7.0, 7.1, and 7.5 seconds on the same platform. The samples are small, but they gave me preliminary evidence that running **TORCS** did not add a large inference delay.

This check does not prove that replay reproduces a complete race. A live strategy changes the car and therefore the states encountered later. Replay is suitable for asking how two prompts

respond to the same fixed situations; it is not a substitute for measuring driving performance. I also implemented `bench_latency.py` so model-server latency could be separated from simulator timing. Without that separation, a slow result could be blamed on the wrong component.

4.4 Discarding measurements that did not answer the question

The final platform test showed that two earlier measurements should not be used.

The first was the `sensitive` column in the prompt-comparison output. It was intended to count decisions that changed after a meaningful change in race state. However, the implementation also treated a change in the explanatory text as a change, even when the strategy remained the same. At temperature 0.2, wording can vary between calls. The perturbation also changed fuel from a high value to 60 L, which was still sufficient for the tested race. The test therefore did not create a state in which a different strategy was clearly expected.

The second was a note in `analyse_bot_trace.py` that described a 4 to 6 second polling cycle. This value had been hardcoded for an earlier configuration and no longer matched the final system. I documented both figures as values that should not be quoted.

4.5 Revising my own conclusion

The most uncomfortable result was the one that contradicted my earlier interpretation. I had described the legacy prompt as showing mode collapse because it produced only one distinct strategy on the selected corpus. When I later compared the variants on the same states, the reasoning prompt also returned `ATTACK` for 48 of 50 cases.

The simpler explanation was the data: the trace came from a trouble-free race where attacking was usually reasonable. Diversity was not the same as quality when the situations themselves were uniform. I removed the mode-collapse claim rather than defend a convenient metric, and this directly motivated the more discriminating corpus proposed in future work.

5 Project Management Skills

5.1 Building the feature from the bottom up

The group divided implementation by feature. I took the bot’s technical foundation and strategy pipeline, which gave my work a natural dependency order. As Table 4 shows, I started with communication and control, then added the track and safety layers, and only then connected Granite. This bottom-up order meant that every new layer rested on something I could already run and inspect.

Table 4: Personal development phases.

Phase	Period	Main work
1	June to 2 July	SCR parser, low-level controller, and pure-pursuit baseline
2	3 to 14 July	Main repository integration, safety filter, and track model
3	15 to 26 July	Granite strategy integration and prompt development
4	27 July to 18 August	Replay traces, display-label tests, reasoning-prompt analysis, and HPC deployment

My weekly routine used Tuesday and Thursday afternoons for concentrated implementation. Monday and Wednesday were mainly used for integration and documentation, and Friday morning was used for review and meeting preparation. This schedule was adjusted when integration work or experiments required longer continuous sessions.

5.2 Using Git as an engineering record

I used Git for more than storing code. Routine commits marked small implementation steps, while commits that changed an experimental conclusion recorded the reason and the affected measurements. I worked on `dev/aibot` and integrated through pull requests, so an incomplete controller did not have to destabilise the group demonstration.

The commit history was useful, but it was not a replacement for evaluation notes. I kept separate documents for the HPC platform comparison and replay experiments, including the configuration, observations, and limitations. My teammate maintained the fault-injection records and the main bot test matrix.

5.3 Explaining decisions to the team and client

I contributed to weekly progress reports for IBM stakeholders, including updates on the Granite integration, middleware changes, and an early-August demonstration video. When presenting the strategy switch, I explained that it controls requests for new model advice but does not disable the local safety controller. This distinction was important because the dashboard could otherwise imply that Granite directly controlled the car. The client response to the demonstration was strongly positive.

Within the group, the clearest disagreement concerned the strategy badge. It appeared static when **ATTACK** remained appropriate for a long period, and changing the model output was suggested as a way to make it look more active. I argued that this would optimise the appearance of intelligence by changing an already measured decision. Rather than asking the team to accept that argument on trust, I proposed the display-label approach and added tests showing that a label cannot change the value returned by the safety filter.

5.4 Managing risk without stopping development

The main risks were simulator instability, model latency, and unsafe interaction between the two timing domains. I reduced the part I controlled through the non-blocking architecture, the initial safety layer, deterministic replay, and the move to BluePebble when local inference became too variable. My teammate led the later fault-injection and broader regression work. The HPC deployment took longer than I expected, but it kept the final prompt and latency tests from depending on an unstable local service.

6 Critical Self-Evaluation and Future Work

6.1 What I would do differently

My main evaluation mistake was collecting the data I could obtain before defining the cases that would answer the question. The recorded corpora mainly contain trouble-free races, precisely the situation in which several prompts can reasonably return **ATTACK**. If I repeated the project, I

would begin by defining low-fuel, high-damage, and close-traffic cases, then collect or construct traces around them.

I also selected one metric because it was easy to compute. Counting distinct strategies did not measure whether the selected strategy was correct. A more useful metric would compare each decision with a defensible reference label, although producing those labels requires manual work and agreement on ambiguous cases. In future evaluations I would write down the expected interpretation and failure mode of each metric before implementing it.

The largest omission is also the experiment a reader is most likely to ask for: the same controller, on the same tracks, with Granite enabled and disabled. I measured latency, prompt behaviour, replay behaviour, and platform differences, but not whether Granite improved lap time, consistency, damage, or off-track frequency. I spent the final week on HPC deployment and the platform comparison. In hindsight, I would run the ablation first.

I also spent too much time on infrastructure relative to the evaluation design. The BluePebble scripts were necessary for stable testing, but the work expanded because the tasks were concrete and easy to verify. Designing a useful corpus was less straightforward, but it was more important to the central research question. I should have reviewed that balance earlier with the supervisor.

6.2 Future work

The first priority is to build a strategy corpus containing states where the expected decision genuinely changes. Examples include crossing a fuel threshold, accumulating enough damage to justify conservative driving, and remaining in close traffic for several control cycles.

The second priority is the strategy-layer ablation described above. It should be repeated on the same tracks and controller configuration so that the effect of Granite can be separated from the quality of the local controller.

The **sensitive** metric should be replaced with a measure based on semantic equivalence and reference labels. An LLM-based judge could be explored, but its decisions should be checked against a manually labelled subset rather than treated as ground truth.

The track model also creates an opportunity to compensate for inference delay. Because Granite responds approximately seven seconds after a request, the prompt could describe a predicted future state rather than only the current one. This would require testing prediction error as well as strategy quality.

Finally, race traces provide pairs of states and model decisions. A smaller model could be trained on these data and compared with Granite. This would test whether the useful part of the strategy behaviour requires a general language model or can be reproduced with a faster specialised model.

6.3 Closing reflection

I expected the hardest part to be connecting a language model to a racing car. In practice, the harder problem was deciding what the model had contributed and building instruments capable of answering that question.

My technical contribution was the bot’s foundation: the SCR protocol layer, the initial controller and safety layer, the original track model, Granite integration, replay and latency tools, HPC deployment, and dashboard strategy labels. Local control remained below the 20 ms target while multi-second model responses stayed outside the driving loop. The integrated bot’s opponent

handling, recovery, pit behaviour, later tuning, and most of the final regression coverage were completed by my teammate and are not claimed as my individual work.

The result is less tidy than a report of uniformly successful experiments. I rejected two measurements, withdrew the mode-collapse interpretation, and did not complete the most direct ablation. I regard that as a limitation of the project but also as evidence that the evaluation process changed my conclusions rather than merely confirming them. The current system is therefore best understood as a functioning basis for the next experiment: repeated, deliberately varied races with Granite enabled and disabled.

References

- [1] Ahn, M. et al. (2022). “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.” *Conference on Robot Learning*.
- [2] Wymann, B., Espie, E., Guionneau, C., Dimitrakakis, C., Coulom, R. and Sumner, A. (2000). *TORCS, The Open Racing Car Simulator*. Available at: <http://torcs.sourceforge.net>.
- [3] Loiacono, D., Cardamone, L. and Lanzi, P. L. (2013). “Simulated Car Racing Championship: Competition Software Manual.” *arXiv:1304.1672*.
- [4] IBM Research (2026). *Granite*. Available at: <https://research.ibm.com/topics/granite>.
- [5] Brooks, R. A. (1986). “A Robust Layered Control System for a Mobile Robot.” *IEEE Journal of Robotics and Automation*, 2(1), pp. 14–23. doi: 10.1109/JRA.1986.1087032.
- [6] Sha, L. R. (1995). *A Software Architecture for Dependable and Evolvable Industrial Computing Systems*. Technical Report CMU/SEI-95-TR-005, Software Engineering Institute, Carnegie Mellon University.